

Projet final : Plateforme de Gestion de Tournois de sport

Description :

Le projet final consiste en la création d'une plateforme de gestion de tournois de sport. L'application permettra aux utilisateurs de créer, organiser et participer à des tournois de sport. Les fonctionnalités principales incluent la gestion des tournois, des joueurs, des inscriptions et des parties.

Caractéristiques

Le projet peut être réalisé par des équipes de 3 personnes maximum.

Rendu

Vous devez soumettre une archive contenant le code source de votre projet. Vous devez également fournir une documentation complète de votre solution et comment l'installer si nécessaire (ceci inclut le générateur de schéma de base de données).

En cas d'informations manquantes (ou de non-fonctionnement), vous perdrez des points.

Éléments notés

Respect des normes RESTful : 2 pts

Base de données : 3 pts

Administration : 5 pts

Sécurité de votre application : 5 pts

Tests unitaires : 2 pts

Respect des spécificités : 3 pts

Jusqu'à 50% des points pour chaque catégorie peuvent être perdus à cause de la qualité du code et/ou de la sécurité. Vous devez travailler comme vous le feriez dans une entreprise ayant un impact sur les clients !

Fonctionnalités clés :

- **Gestion des Tournois** : Les utilisateurs peuvent créer et organiser des tournois de sport, définir les dates, les lieux et les règles des tournois.
- **Gestion des Joueurs** : Les utilisateurs peuvent s'inscrire, créer un profil et participer à des tournois en tant que participant.
- **Inscriptions aux Tournois** : Les participants peuvent s'inscrire à des tournois disponibles, gérer leurs inscriptions et suivre leur statut.
- **Gestion des Parties** : Les parties entre les participants sont organisées et gérées au sein des tournois, avec la possibilité de saisir les scores et de déterminer les gagnants.
- **Administration** : Une interface d'administration permet aux administrateurs de gérer les tournois, les participants, les inscriptions et les parties de manière centralisée.

Spécificités :

- Événements :
 - Lorsqu'un tournoi est remporté, une notification doit être envoyée aux participants du tournoi avec le vainqueur dedans.
 - Lorsqu'un participant met à jour son score, une notification doit être envoyée à l'autre joueur pour qu'il le remplisse s'il ne l'a pas encore fait. Lorsqu'un admin modifie les scores, aucune notification n'est envoyée.
 - Lorsque les deux participants ont rempli leur score, le statut de la partie doit automatiquement passer en terminer.
 - L'interface d'administration ne doit être accessible qu'aux admins, elle permettra de gérer les tournois, les parties, les utilisateurs ainsi que les inscriptions à un tournoi.
- Règles :
 - Lorsqu'une partie est créée dans un tournoi, il ne peut faire jouer que des joueurs ayant une inscription confirmée dans celui-ci.
- Commande Symfony :
 - Créer des fixtures.
 - Faire une commande qui prend en entrée un ID utilisateur et qui sort le nombre de victoires et de défaites au total. Rajouter un argument ID de tournoi qui s'il est passé ressort le nombre de victoires et de défaites pour ce tournoi.

Entités

- Tournament (Tournoi) :
 - tournamentName (string) : Nom du tournoi
 - startDate (date) : Date de début du tournoi
 - endDate (date) : Date de fin du tournoi
 - location (string, optionnel) : Lieu du tournoi
 - description (texte) : Description du tournoi
 - maxParticipants (integer) : Nombre maximum de participants au tournoi
 - status (dynamique) : Le statut du tournoi doit être renvoyé dynamiquement en se basant sur les dates de début et de fin du tournoi.
 - sport (string) : Nom du sport
 - organizer (relation vers l'entité User) : Organisateur du tournoi
 - winner (relation vers l'entité User) : Gagnant du tournoi
 - Attention la relation avec Game est au pluriel (games)
- User (Joueur) :
 - lastName (string) : Nom de famille du joueur
 - firstName (string) : Prénom du joueur
 - username (string) : Nom d'utilisateur du joueur
 - emailAddress (string) : Adresse email du joueur
 - password (string) : Mot de passe du joueur (haché)
 - status (string) : Statut de l'utilisateur (actif, suspendu, banni)
- Registration (Inscription) :
 - player (relation vers l'entité User) : Joueur inscrit au tournoi
 - tournament (relation vers l'entité Tournament) : Tournoi auquel le joueur est inscrit
 - registrationDate (date) : Date d'inscription du joueur au tournoi
 - status (string) : Statut de l'inscription (confirmée, en attente)
- SportMatch (Partie) :
 - tournament (relation vers l'entité Tournament) : Tournoi auquel le match appartient
 - player1 (relation vers l'entité User) : Joueur 1 du match
 - player2 (relation vers l'entité User) : Joueur 2 du match
 - matchDate (date) : Date du match
 - scorePlayer1 (integer) : Score du joueur 1 dans le match
 - scorePlayer2 (integer) : Score du joueur 2 dans le match
 - status (string) : Statut du match (en attente, en cours, terminé)

Routes

1. Tournament Management (Gestion des tournois) :

- GET /api/tournaments : récupère la liste des tournois
 - Contraintes : Aucune.
- POST /api/tournaments : crée un nouveau tournoi
 - Contraintes : Les données du tournoi doivent être fournies dans le corps de la requête au format JSON. Au moins, le nom du tournoi, la date de début, la date de fin et la description doivent être fournis.
- GET /api/tournaments/{id} : récupère les détails d'un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant.
- PUT /api/tournaments/{id} : mets à jour les informations d'un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant. Les données mises à jour du tournoi doivent être fournies dans le corps de la requête au format JSON.
- DELETE /api/tournaments/{id} : Supprime un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant.

2. Player Management (Gestion des joueurs) :

- GET /api/players : récupère la liste des joueurs
 - Contraintes : Aucune.
- POST /register : créer un utilisateur
 - Contraintes : Les données de l'utilisateur doivent être fournies dans le corps de la requête au format JSON. Au moins, le nom, le prénom, le nom d'utilisateur, l'adresse e-mail et le mot de passe doivent être fournis.
- GET /api/players/{id} : récupère les détails d'un joueur spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un joueur existant.
- PUT /api/players/{id} : mets à jour les informations d'un joueur spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un joueur existant. Les données mises à jour du joueur doivent être fournies dans le corps de la requête au format JSON.
- DELETE /api/players/{id} : Supprime un joueur spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un joueur existant.

3. Registration Management (Gestion des inscriptions) :

- GET /api/tournaments/{id}/registrations : récupère la liste des inscriptions pour un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant.
- POST /api/tournaments/{id}/registrations : inscris un joueur à un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant. Les données de l'inscription doivent être fournies dans le corps de la requête au format JSON, contenant l'identifiant du joueur à inscrire.
- DELETE /api/tournaments/{idTournament}/registrations/{idRegistration} : Annule l'inscription d'un joueur à un tournoi spécifique

- contrainte : {idTournament} doit correspondre à un identifiant valide d'un tournoi existant. {idRegistration} doit correspondre à un identifiant valide d'une inscription existante pour ce tournoi.

4. SportMatch Management (Gestion des parties) :

- GET /api/tournaments/{id}/sport-matches : récupère la liste des parties pour un tournoi spécifique
 - contrainte : {id} doit correspondre à un identifiant valide d'un tournoi existant.
- POST /api/tournaments/{id}/sport-matches : crée une nouvelle partie pour un tournoi spécifique
 - Contraintes : {id} doit correspondre à un identifiant valide d'un tournoi existant. Les données du match doivent être fournies dans le corps de la requête au format JSON, contenant les identifiants des joueurs participants.
- GET /api/tournaments/{idTournament}/sport-matches/{idSportMatches} : récupère les détails d'une partie spécifique
 - Contraintes : {idTournament} doit correspondre à un identifiant valide d'un tournoi existant. {idSportMatches} doit correspondre à un identifiant valide d'une partie pour ce tournoi.
- PUT /api/tournaments/{idTournament}/sport-matches/{idSportMatches} : mets à jour les résultats d'une partie spécifique
 - Contraintes : {idTournament} doit correspondre à un identifiant valide d'un tournoi existant. {idSportMatches} doit correspondre à un identifiant valide d'une partie pour ce tournoi. Les données mises à jour de la partie doivent être fournies dans le corps de la requête au format JSON, contenant les scores des joueurs.
 - Attention lors de la mise à jour des scores, seul le joueur concerné peut mettre son score à jour, ou une admin.
- DELETE /api/tournaments/{idTournament}/sport-matches/{idSportMatches} : Supprime une partie spécifique
 - Contraintes : {idTournament} doit correspondre à un identifiant valide d'un tournoi existant. {idSportMatches} doit correspondre à un identifiant valide d'une partie pour ce tournoi.